Turing Pi 2.5 RK1 HA Kubernetes Cluster

Dual-Stack IPv4/IPv6 Bare Metal Deployment Guide

4x RK1 (RK3588) 16GB Nodes • 1TB NVMe Each • Ubuntu 24.04

K3s • Embedded etcd • kube-vip • MetalLB • Longhorn • Rancher

October 2026 — Version 1.18

Table of Contents

- 1. Installing Ubuntu 24.04 via BMC
  - 1.1 Prerequisites
  - 1.2 Flash Ubuntu to eMMC via BMC Web UI
  - 1.3 Alternative: Flash via BMC CLI (tpi command)
  - 1.4 Initial Login and System Update
  - 1.5 Clone eMMC to NVMe
  - 1.6 Configure Boot Priority
  - 1.7 Expand the NVMe Root Partition
  - 1.8 Restore Ping for Non-Root Users
  - 1.9 Install mDNS Support (Avahi)
  - 1.10 Repeat for All Nodes
- 2. Architecture Overview
  - 2.1 Node Assignment
  - 2.2 Network Architecture
  - 2.3 Component Summary
- 3. Host Preparation (All 4 Nodes)
  - 3.1 Set Hostname
  - 3.2 Configure Dual-Stack Static Networking
  - 3.3 Configure /etc/hosts
  - 3.4 Disable Swap
  - 3.5 Load Kernel Modules
  - 3.6 Set Kernel Parameters for Dual-Stack
  - 3.7 Update System Packages
  - 3.8 Verify Connectivity
- 4. K3s HA Cluster Deployment
  - 4.1 Node 1 — Initialize the Cluster
  - 4.2 Node 2 — Join as Second Server
  - 4.3 Node 3 — Join as Third Server
  - 4.4 Node 4 — Join as Worker
  - 4.5 Verify etcd Cluster Health

    - 4.6 Optional: Install etcdctl
- 5. Kube-VIP — Floating Virtual IP
    - 5.1 Deploy RBAC
    - 5.2 Deploy IPv4 VIP DaemonSet
    - 5.3 Verify VIP
        - 5.3.1 IPv6 kubeconfig with Multi-Node Failover
        - 5.3.2 Switching Between IPv6 Nodes
    - 5.4 Update Worker Node to Use VIP
    - 5.5 Configure kubectl Access on All Nodes
- 6. MetalLB — Dual-Stack Load Balancer
    - 6.1 Install MetalLB
    - 6.2 Configure Dual-Stack IP Pool
- 7. Nginx Ingress Controller
    - 7.1 Install Helm
    - 7.2 Deploy Nginx Ingress with Dual-Stack
- 8. Longhorn — Distributed NVMe Storage
    - 8.1 Install Longhorn
    - 8.2 Set as Default StorageClass
    - 8.3 Access Longhorn UI
- 9. Management Dashboard
    - 9.1 Option A: Rancher
    - 9.2 Option B: Portainer
- 10. Remote Kubeconfig Setup
    - 10.1 Configure kubectl on Remote Workstation
- 11. Cluster Validation
    - 11.1 Verify All Nodes
    - 11.2 Verify Dual-Stack Pod Networking
    - 11.3 Verify HA Failover
    - 11.4 Verify Services Get Dual-Stack IPs
- 12. Optional: Prometheus + Grafana Monitoring
    - 12.1 Install kube-prometheus-stack
    - 12.2 Access Grafana
    - 12.3 Recommended Dashboards
    - 12.4 Add Longhorn Monitoring
    - 12.5 Set a Home Dashboard
- 13. Maintenance and Management
    - 13.1 Useful Commands
    - 13.2 Taking a Single Node Out of Service

    - 13.3 Graceful Full Cluster Shutdown
    - 13.4 Restarting the Cluster After Power-Down
    - 13.5 K3s Upgrades
    - 13.6 Backups
    - 13.7 Uninstalling K3s
- 14. Architecture Diagram

---

1. Installing Ubuntu 24.04 via BMC

Before configuring the cluster, Ubuntu 24.04 Server must be flashed to each RK1 node's eMMC via the Turing Pi 2.5 BMC, then cloned to the NVMe drive for production use. The eMMC is the RK1's built-in storage and is used for initial flashing; the 1TB NVMe provides significantly better I/O performance for Kubernetes workloads.

1.1 Prerequisites

Ensure the following before starting:

BMC Firmware: The Turing Pi 2.5 BMC must be running firmware version 2.x or later. Check and upgrade via the BMC web interface if needed. See the Turing Pi documentation for firmware upgrade instructions.

Ubuntu Image: Download the Ubuntu 24.04 Server image for the Turing RK1 from the official firmware site at https://firmware.turingpi.com/turing-rk1/ or from Joshua Riek's Ubuntu Rockchip releases at https://joshua-riek.github.io/ubuntu-rockchip-download/boards/turing-rk1.html. Use the server image (not desktop). The file will be named similar to ubuntu-24.04-preinstalled-server-arm64-turing-rk1.img.xz.

Default Credentials: The server image ships with login 'ubuntu' and password 'ubuntu'. You will be prompted to change the password on first login.

1.2 Flash Ubuntu to eMMC via BMC Web UI

Repeat the following process for each of the four nodes (Nodes 1 through 4). Only one node can be flashed at a time.

Step 1: Open the BMC web interface by navigating to the BMC's IP address in your browser (this is typically the Turing Pi board's management IP).

Step 2: Navigate to the 'Flash Node' page in the BMC UI.

Step 3: Select the target node (Node 1, 2, 3, or 4) from the dropdown.

Step 4: Choose the downloaded Ubuntu 24.04 .img.xz image file. The BMC accepts both compressed (.img.xz) and uncompressed (.img) files.

Step 5: Click 'Install OS' and confirm when prompted.

Step 6: Wait for flashing to complete. The BMC flashes at approximately 8 minutes per 1GB of image size, plus additional time for verification. A typical server image takes approximately 30–60 minutes per node.

Once flashing completes, the node will automatically reboot into the freshly installed Ubuntu. The progress bar may jump to 100% early — this is a known cosmetic issue and does not affect the actual flashing process. Wait for the verification step to complete before proceeding.

1.3 Alternative: Flash via BMC CLI (tpi command)

If you prefer the command line, you can use the tpi tool from another machine or SSH into the BMC directly. This is useful for scripting the flash process across all four nodes. An SD card must be installed in the BMC for this method.

```
# Copy the image to the BMC's SD card
scp ubuntu-24.04-preinstalled-server-arm64-turing-rk1.img.xz \
  root@<bmc-ip>:/mnt/sdcard/

# SSH into the BMC
ssh root@<bmc-ip>

# Flash each node (power off first, then flash)
tpi power -n 1 off
tpi flash -i /mnt/sdcard/ubuntu-24.04-preinstalled-server-arm64-turing-rk1.img.xz -n 1

tpi power -n 2 off
tpi flash -i /mnt/sdcard/ubuntu-24.04-preinstalled-server-arm64-turing-rk1.img.xz -n 2

tpi power -n 3 off
tpi flash -i /mnt/sdcard/ubuntu-24.04-preinstalled-server-arm64-turing-rk1.img.xz -n 3

tpi power -n 4 off
tpi flash -i /mnt/sdcard/ubuntu-24.04-preinstalled-server-arm64-turing-rk1.img.xz -n 4
```

1.4 Initial Login and System Update

After each node boots from eMMC, connect via SSH or the BMC's UART console. Each node will receive a DHCP address from your network — check your router or use the BMC interface to find the assigned IP.

```
# SSH into the node (default credentials: ubuntu / ubuntu)
ssh ubuntu@<node-ip>

# You will be prompted to change the default password on first login

# Update all packages to ensure latest firmware and kernel patches
sudo apt update && sudo apt upgrade -y

# Reboot to apply kernel updates
sudo reboot
```

Important: Run apt update and upgrade on each node before proceeding to the NVMe migration. This ensures you have the latest kernel, device tree, and u-boot fixes which are critical for reliable NVMe boot support on the RK1.

1.5 Clone eMMC to NVMe

The RK1 includes a built-in utility called ubuntu-rockchip-install that properly clones the running eMMC system to the NVMe drive. This is the recommended method because it automatically assigns new partition UUIDs to the NVMe, preventing boot conflicts between the eMMC and NVMe. Do not use dd to copy the image directly — identical UUIDs on both drives will cause u-boot to randomly mount partitions from either device.

Run the following on each node after the system update and reboot:

```
# Verify the NVMe drive is detected
lsblk
# You should see nvme0n1 listed (your 1TB NVMe)

# Clone the running eMMC system to the NVMe
sudo ubuntu-rockchip-install /dev/nvme0n1
```

This command partitions the NVMe, copies both the boot and root partitions from eMMC, assigns new UUIDs, and updates /etc/fstab on the NVMe's root filesystem. The process takes a few minutes per node.

1.6 Configure Boot Priority

The RK1's u-boot boot order prioritizes NVMe over eMMC by default. After running ubuntu-rockchip-install, simply reboot and the system should boot from the NVMe automatically:

```
# Reboot to boot from NVMe
sudo reboot

# After reboot, verify you are running from NVMe
lsblk
# The root filesystem (/) should be mounted on nvme0n1p2
# and the boot partition (/boot/firmware) on nvme0n1p1

df -h /
# Should show /dev/nvme0n1p2 as the device
```

If the system still boots from eMMC, install u-boot to the SPI flash to ensure NVMe boot priority:

```
# Install u-boot to SPI flash (ensures NVMe is prioritized)
sudo u-boot-install-mtd /dev/mtdblock0

# Reboot and verify again
sudo reboot
```

1.7 Expand the NVMe Root Partition

The ubuntu-rockchip-install command clones the eMMC partitions at their original size, leaving the majority of your 1TB NVMe unallocated. Expand the root partition to use the full drive:

```
# Install growpart if not already present
sudo apt install -y cloud-guest-utils

# Expand the root partition to fill the NVMe
sudo growpart /dev/nvme0n1 2

# Resize the filesystem
sudo resize2fs /dev/nvme0n1p2

# Verify the full 1TB is now available
df -h /
# Should show close to 1TB total size
```

1.8 Restore Ping for Non-Root Users

Ubuntu 24.04 no longer grants the ping binary the capability to send raw packets by default, so ping will fail for non-root users. Rather than prefixing every ping command with sudo, restore the capability once on each node:

```
sudo setcap cap_net_raw+p /usr/bin/ping

# Verify it works without sudo
ping -c 1 127.0.0.1
```

This persists across reboots and has no meaningful security impact — ping is a basic diagnostic tool.

1.9 Install mDNS Support (Avahi)

Install Avahi on each node to enable mDNS (multicast DNS) hostname resolution. This allows nodes to be reached via \<hostname>.local addresses (e.g., k3-node1.local) without relying on a local DNS server or /etc/hosts entries for discovery. This is especially useful during initial setup and for accessing services from workstations on the same network.

```
# Install Avahi daemon and utilities
sudo apt install -y avahi-daemon avahi-utils

# Enable and start the service
sudo systemctl enable avahi-daemon
sudo systemctl start avahi-daemon

# Verify mDNS is working (from another node or workstation)
# At this stage the hostname is still 'ubuntu', so use:
avahi-resolve -n ubuntu.local
ping ubuntu.local
# After hostnames are set in Section 3.1, use k3-node1.local etc.
```

By default, Avahi advertises the system hostname set via hostnamectl. At this stage the hostname is still the default 'ubuntu', so mDNS will advertise as ubuntu.local. Once hostnames are configured in Section 3.1 (e.g., k3-node1), Avahi will automatically advertise the updated names: k3-node1.local, k3-node2.local, k3-node3.local, and k3-node4.local.

Avahi also supports IPv6 mDNS out of the box, so nodes will be resolvable over both IPv4 and IPv6 once dual-stack networking is configured.

To enable mDNS resolution on the nodes themselves (so they can resolve other .local addresses), ensure the NSS mDNS module is installed:

```
# Install NSS mDNS module for .local resolution
sudo apt install -y libnss-mdns

# Verify /etc/nsswitch.conf includes mdns
# It should contain a line like:
#   hosts: files mdns4_minimal [NOTFOUND=return] dns
cat /etc/nsswitch.conf | grep hosts
```

If the hosts line does not include mdns4_minimal, edit /etc/nsswitch.conf:

```
# Edit nsswitch.conf if needed
sudo sed -i 's/^hosts:.*/hosts: files mdns4_minimal [NOTFOUND=return] dns/' \
/etc/nsswitch.conf
```

1.10 Repeat for All Nodes

Complete sections 1.4 through 1.9 for each of the four nodes before proceeding to host preparation. Once all nodes are booting from NVMe with Avahi running, the eMMC serves as a recovery fallback — if an NVMe ever fails, the node can still boot from eMMC.

At this point all four nodes should be running Ubuntu 24.04 from their 1TB NVMe drives with the full disk space available.

2. Architecture Overview

This guide walks through the complete setup of a high-availability Kubernetes cluster running on four Turing Pi 2.5 RK1 nodes with dual-stack IPv4/IPv6 networking. The deployment uses K3s with embedded etcd for a lightweight, ARM64-native Kubernetes distribution that supports full HA with automatic failover.

The cluster architecture uses three server (control plane) nodes running embedded etcd for quorum-based consensus, and one dedicated worker node. All four nodes run workloads by default, providing 64GB of total cluster memory and 4TB of NVMe storage.

2.1 Node Assignment

Node	IPv4	IPv6 (ULA)	Role	Hostname
1	192.168.4.101	fd00::101	Server + etcd	k3-node1
2	192.168.4.102	fd00::102	Server + etcd	k3-node2
3	192.168.4.103	fd00::103	Server + etcd	k3-node3
4	192.168.4.104	fd00::104	Worker (agent)	k3-node4

2.2 Network Architecture

The cluster uses the following CIDR ranges for dual-stack networking:

```
Pod IPv4 CIDR: 10.42.0.0/16
Pod IPv6 CIDR: fd10:42::/56
Service IPv4 CIDR: 10.43.0.0/16
Service IPv6 CIDR: fd10:43::/112
MetalLB IPv4 Pool: 192.168.4.200 - 192.168.4.220
MetalLB IPv6 Pool: fd00::200 - fd00::220
Control Plane VIP: 192.168.4.100 (IPv4 only — see Section 5.3 for IPv6 note)
```

2.3 Component Summary

Component	Solution	Purpose
OS	Ubuntu 24.04	Pre-installed on all nodes
K8s Distribution	K3s (HA embedded etcd)	Lightweight, ARM64-native
Networking	Flannel dual-stack	IPv4 + IPv6 pod networking
Control Plane VIP	kube-vip	Floating IPv4 VIP (.100)
Load Balancer	MetalLB (L2 mode)	Dual-stack external IPs
Ingress	Nginx Ingress Controller	Dual-stack HTTP/S routing
Storage	Longhorn	Distributed NVMe storage
Dashboard	Rancher / Portainer	Cluster management UI
Monitoring	Prometheus + Grafana	Optional observability stack

3. Host Preparation (All 4 Nodes)

Perform all steps in this section on every node unless otherwise noted.

3.1 Set Hostname

Run the appropriate command on each node:

```
sudo hostnamectl set-hostname k3-node1 # On node 1
sudo hostnamectl set-hostname k3-node2 # On node 2
sudo hostnamectl set-hostname k3-node3 # On node 3
sudo hostnamectl set-hostname k3-node4 # On node 4
```

3.2 Configure Dual-Stack Static Networking

The RK1's Ethernet interface can appear as end0 or end1 unpredictably between reboots depending on kernel device enumeration order. To avoid this problem, the netplan configuration uses MAC address matching instead of a hardcoded interface name, and renames the interface to a stable 'eth0'.

Important — cloud-init conflict: Ubuntu on the RK1 ships with a cloud-init netplan file at /etc/netplan/50-cloud-init.yaml that uses wildcard interface matching (en*, eth*) with DHCP enabled. This file will conflict with your static IP configuration because it matches the same interface. You must remove this file before applying the static config, otherwise DHCP will override your static IP assignment, which causes etcd peer connection failures and prevents the node from joining the cluster.

```
# Remove the cloud-init config to avoid DHCP conflicts
sudo rm -f /etc/netplan/50-cloud-init.yaml

# Auto-detect the MAC address of the Ethernet interface
MAC_ADDR=$(ip link show | grep -A1 "end[01]:" | awk '/link\/ether/ {print $2}')
echo "Detected MAC address: ${MAC_ADDR}"

# Write the netplan config with a placeholder
sudo tee /etc/netplan/01-network.yaml << 'EOF'
network:
  version: 2
  ethernets:
    eth0:
      match:
        macaddress: "PLACEHOLDER_MAC"
      set-name: eth0
      addresses:
        - 192.168.4.101/22       # Change per node: .101, .102, .103, .104
        - fd00::101/64           # Change per node: ::101, ::102, ::103, ::104
      routes:
        - to: default
          via: 192.168.4.1
        # No static IPv6 default route: the IPv6 default route comes from
        # router advertisements (RA). Do NOT add "::/0 via fd00::1".
      nameservers:
        addresses:
          - 192.168.4.1
          - "2606:4700:4700::1111"
EOF

# Replace the placeholder with the detected MAC address
sudo sed -i "s/PLACEHOLDER_MAC/${MAC_ADDR}/" /etc/netplan/01-network.yaml

# Verify the MAC was inserted correctly
sudo grep macaddress /etc/netplan/01-network.yaml
```

The script auto-detects the MAC address from whichever interface name (end0 or end1) is active, writes the netplan config, then uses sed to insert the MAC. The 'match: macaddress' directive ensures the config always finds the correct physical interface regardless of kernel naming, and 'set-name: eth0' renames it to a stable, predictable name.

IPv6 default route (pitfall): Earlier versions of this template had a static route `::/0 via fd00::1` and `fd00::1` as a DNS server. On this LAN no host owns fd00::1, so that route is a dead gateway: alongside

the RA-learned route it creates an ECMP default, and replies to about half of external IPv6 hosts are blackholed. It was removed from all nodes in September 2026. Let the IPv6 default route come from router advertisements.

Before applying, edit the addresses line to match this node's assignment (.101, .102, .103, or .104). Then apply:

```
sudo netplan apply

# Verify the interface is now named eth0 with the correct IPs
ip -4 addr show eth0
ip -6 addr show eth0
```

3.3 Configure /etc/hosts

Add all node entries on every node:

```
cat <<EOF | sudo tee -a /etc/hosts
# IPv4
192.168.4.100 k3s-vip
192.168.4.101 k3-node1
192.168.4.102 k3-node2
192.168.4.103 k3-node3
192.168.4.104 k3-node4

# IPv6
fd00::101 k3-node1-v6
fd00::102 k3-node2-v6
fd00::103 k3-node3-v6
fd00::104 k3-node4-v6
EOF
```

3.4 Disable Swap

```
sudo swapoff -a
sudo sed -i '/ swap / s/^/#/' /etc/fstab
```

3.5 Load Kernel Modules

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
ip6_tables
ip6table_filter
ip6table_nat
EOF

sudo modprobe overlay br_netfilter ip6_tables ip6table_filter ip6table_nat
```

3.6 Set Kernel Parameters for Dual-Stack

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s-dualstack.conf
# IPv4
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
# IPv6
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv6.conf.all.forwarding = 1
net.ipv6.conf.default.forwarding = 1
# Ensure IPv6 is enabled
net.ipv6.conf.all.disable_ipv6 = 0
net.ipv6.conf.default.disable_ipv6 = 0
net.ipv6.conf.lo.disable_ipv6 = 0
EOF
sudo sysctl --system
```

3.7 Update System Packages

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl jq open-iscsi nfs-common
```

The open-iscsi and nfs-common packages are required by Longhorn for distributed storage.

Required: blacklist Longhorn devices in multipathd. Ubuntu runs `multipathd`, and it claims Longhorn's iSCSI block devices (vendor `IET`, product `VIRTUAL-DISK`). The volume then can't be mounted (`Can't open blockdev` in dmesg) or fails with EIO on writes. On this cluster that caused a pod to crashloop for 35 days. Add the blacklist on every node before installing Longhorn:

```
cat <<'EOF' | sudo tee /etc/multipath.conf
defaults {
    user_friendly_names yes
}

blacklist {
    device {
        vendor "IET"
        product "VIRTUAL-DISK"
    }
}
EOF
sudo systemctl restart multipathd
```

3.8 Verify Connectivity

```
# Test IPv4 between nodes
ping -c 2 192.168.4.102

# Test IPv6 between nodes
ping6 -c 2 fd00::102
```

4. K3s HA Cluster Deployment

K3s is deployed with embedded etcd across three server nodes for high availability. The cluster uses dual-stack networking with Flannel as the CNI. Nodes must be joined one at a time — wait for each to reach Ready status before adding the next.

Version pinning: every install command below pins `INSTALL_K3S_VERSION=v1.37.1+k3s1`, the version this cluster runs (as of October 2026). Without the pin, get.k3s.io installs whatever is the current stable release. Use the same version on every node, and keep it in step with the system-upgrade Plans (Section 13.5).

Note on kubectl during setup: K3s generates its kubeconfig at /etc/rancher/k3s/k3s.yaml which is only readable by root. The kubectl commands in this section use 'export KUBECONFIG=/etc/rancher/k3s/k3s.yaml' combined with 'sudo -E' (which preserves environment variables) so that kubectl can locate the config file. This is set up once on k3-node1 after K3s initializes, and the exported variable persists for the remainder of the shell session. Non-root kubectl access for all nodes is configured later in Section 5.5.

4.1 Node 1 — Initialize the Cluster

Create the K3s configuration file:

```
sudo mkdir -p /etc/rancher/k3s

cat <<'EOF' | sudo tee /etc/rancher/k3s/config.yaml
# First server — initializes the HA cluster
cluster-init: true

node-name: k3-node1
write-kubeconfig-mode: "0644"

# TLS SANs for all access points
tls-san:
  - "192.168.4.100"
  - "192.168.4.101"
  - "192.168.4.102"
  - "192.168.4.103"
  - "fd00::101"
  - "fd00::102"
  - "fd00::103"
  - "k3s-vip"

# Dual-stack CIDRs
cluster-cidr: "10.42.0.0/16,fd10:42::/56"
service-cidr: "10.43.0.0/16,fd10:43::/112"

# Flannel dual-stack masquerade
flannel-ipv6-masq: true

# Disable built-in LB and ingress (MetalLB + Nginx used instead)
disable:
  - traefik
  - servicelb

# Automated etcd snapshots
etcd-snapshot-schedule-cron: "0 */6 * * *"
etcd-snapshot-retention: 10
EOF
```

Install K3s:

```
curl -sfL https://get.k3s.io | INSTALL_K3S_VERSION=v1.37.1+k3s1 sh -

# Wait for K3s to start and generate the kubeconfig
echo "Waiting for K3s to initialize..."
until [ -f /etc/rancher/k3s/k3s.yaml ]; do
  sleep 2
done
echo "K3s kubeconfig created."

# Export KUBECONFIG for this session so kubectl finds the config
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml

# Wait for the node to be ready
sudo -E kubectl wait --for=condition=Ready node/k3-node1 --timeout=180s

# Verify
sudo -E kubectl get nodes

# Save the join token for the other nodes
sudo cat /var/lib/rancher/k3s/server/node-token
```

Important: Copy the token output. You will need it for all remaining nodes.

4.2 Node 2 — Join as Second Server

Wait until k3-node1 shows Ready, then configure and install on k3-node2:

```
sudo mkdir -p /etc/rancher/k3s

cat <<'EOF' | sudo tee /etc/rancher/k3s/config.yaml
server: https://192.168.4.101:6443
token: <PASTE_TOKEN_FROM_NODE1>

node-name: k3-node2
write-kubeconfig-mode: "0644"

tls-san:
  - "192.168.4.100"
  - "192.168.4.101"
  - "192.168.4.102"
  - "192.168.4.103"
  - "fd00::101"
  - "fd00::102"
  - "fd00::103"
  - "k3s-vip"

cluster-cidr: "10.42.0.0/16,fd10:42::/56"
service-cidr: "10.43.0.0/16,fd10:43::/112"
flannel-ipv6-masq: true

disable:
  - traefik
  - servicelb

etcd-snapshot-schedule-cron: "0 */6 * * *"
etcd-snapshot-retention: 10
EOF

curl -sfL https://get.k3s.io | INSTALL_K3S_VERSION=v1.37.1+k3s1 sh -
```

Verify from k3-node1:

```
sudo -E kubectl get nodes
# Wait until k3-node2 shows Ready before proceeding
```

4.3 Node 3 — Join as Third Server

Identical to k3-node2 but change node-name to k3-node3:

```
sudo mkdir -p /etc/rancher/k3s

cat <<'EOF' | sudo tee /etc/rancher/k3s/config.yaml
server: https://192.168.4.101:6443
token: <PASTE_TOKEN_FROM_NODE1>

node-name: k3-node3
write-kubeconfig-mode: "0644"

tls-san:
  - "192.168.4.100"
  - "192.168.4.101"
  - "192.168.4.102"
  - "192.168.4.103"
  - "fd00::101"
  - "fd00::102"
  - "fd00::103"
  - "k3s-vip"

cluster-cidr: "10.42.0.0/16,fd10:42::/56"
service-cidr: "10.43.0.0/16,fd10:43::/112"
flannel-ipv6-masq: true

disable:
  - traefik
  - servicelb

etcd-snapshot-schedule-cron: "0 */6 * * *"
etcd-snapshot-retention: 10
EOF

curl -sfL https://get.k3s.io | INSTALL_K3S_VERSION=v1.37.1+k3s1 sh -
```

Verify all three server nodes from k3-node1:

```
sudo -E kubectl get nodes
# All three should show Ready with role: control-plane,etcd
```

4.4 Node 4 — Join as Worker

The worker joins through the VIP address (configured in the next section). Until kube-vip is deployed, point at k3-node1 directly:

```
# On k3-node4:
sudo mkdir -p /etc/rancher/k3s

cat <<'EOF' | sudo tee /etc/rancher/k3s/config.yaml
server: https://192.168.4.101:6443
token: <PASTE_TOKEN_FROM_NODE1>
node-name: k3-node4
EOF

curl -sfL https://get.k3s.io | INSTALL_K3S_VERSION=v1.37.1+k3s1
K3S_URL=https://192.168.4.101:6443 \
  K3S_TOKEN="<PASTE_TOKEN_FROM_NODE1>" sh -

# Verify the agent is running
sudo systemctl status k3s-agent
```

The output should show the agent as active (running). Key lines to look for:

```
● k3s-agent.service - Lightweight Kubernetes
     Loaded: loaded (/etc/systemd/system/k3s-agent.service; enabled; preset: enabled)
     Active: active (running) since ...
   Main PID: 12345 (k3s-agent)
```

If the status shows 'active (running)', the agent has successfully connected to the control plane. You can also check the agent logs for confirmation:

```
# Check agent logs for successful connection
sudo journalctl -u k3s-agent --no-pager | tail -20
# Look for: "Successfully connected to server"
# or: "Tunnel established" and no repeated error messages
```

Note: kubectl will not work on k3-node4 at this stage. The K3s agent does not generate a kubeconfig file — this is normal. Kubectl access on the worker node is configured later in Section 5.5.

Verify the full cluster from k3-node1 (switch back to your k3-node1 SSH session):

```
# Run on k3-node1 (where KUBECONFIG is already exported)
sudo -E kubectl get nodes -o wide
# Expected output (IPs and versions may vary):
# NAME STATUS ROLES AGE VERSION INTERNAL-IP
# k3-node1 Ready control-plane,etcd 30m v1.37.1+k3s1 192.168.4.101
# k3-node2 Ready control-plane,etcd 20m v1.37.1+k3s1 192.168.4.102
# k3-node3 Ready control-plane,etcd 10m v1.37.1+k3s1 192.168.4.103
# k3-node4 Ready <none> 2m v1.37.1+k3s1 192.168.4.104
```

All four nodes should show 'Ready' status. The worker node k3-node4 will show '\<none>' under ROLES, which is normal for an agent node. If k3-node4 shows 'NotReady', wait a minute and try again — the agent may still be initializing.

4.5 Verify etcd Cluster Health

K3s embeds etcd directly into its server process but does not include the etcdctl command-line tool. You can verify etcd health using the Kubernetes API and curl without installing anything extra. Run these on k3-node1:

```
# Check etcd health via the Kubernetes API
sudo -E kubectl get --raw=/healthz/etcd
# Expected output: ok

# Check that all server nodes show the etcd role
sudo -E kubectl get nodes
# All three server nodes should show roles: control-plane,etcd

# Query the etcd API directly using curl and the K3s TLS certs
sudo curl -s --cacert /var/lib/rancher/k3s/server/tls/etcd/server-ca.crt \
  --cert /var/lib/rancher/k3s/server/tls/etcd/server-client.crt \
  --key /var/lib/rancher/k3s/server/tls/etcd/server-client.key \
  https://127.0.0.1:2379/health
# Expected output: {"health":"true","reason":""}

# Check the etcd version (confirms the endpoint is responding)
sudo curl -s --cacert /var/lib/rancher/k3s/server/tls/etcd/server-ca.crt \
  --cert /var/lib/rancher/k3s/server/tls/etcd/server-client.crt \
  --key /var/lib/rancher/k3s/server/tls/etcd/server-client.key \
  https://127.0.0.1:2379/version
# Expected output on k3s v1.37 (etcd 3.7.x), e.g.: {"etcdserver":"3.7.x-
k3s1","etcdcluster":"3.7.0"}
```

4.6 Optional: Install etcdctl

If you want the full etcdctl CLI for detailed member listing, health checks, and maintenance tasks, you must install it manually. K3s does not bundle it. Since the RK1 is ARM64, download the arm64 build:

```
# Query the embedded etcd version to match the etcdctl release
ETCD_VER=$(sudo curl -s --cacert /var/lib/rancher/k3s/server/tls/etcd/server-ca.crt \
  --cert /var/lib/rancher/k3s/server/tls/etcd/server-client.crt \
  --key /var/lib/rancher/k3s/server/tls/etcd/server-client.key \
  https://127.0.0.1:2379/version | jq -r '.etcdserver' | sed 's/-k3s1//')

# Download and install etcdctl for arm64
curl -sL https://github.com/etcd-io/etcd/releases/download/v${ETCD_VER}/etcd-
v${ETCD_VER}-linux-arm64.tar.gz \
  | sudo tar xzf - -C /usr/local/bin --strip-components=1 etcd-v${ETCD_VER}-linux-
arm64/etcdctl

# Verify installation
etcdctl version
```

Set up environment variables to simplify etcdctl usage. Add these to ~/.bashrc on each server node:

```
cat <<'EOF' >> ~/.bashrc
export ETCDCTL_API=3
export ETCDCTL_ENDPOINTS='https://127.0.0.1:2379'
export ETCDCTL_CACERT='/var/lib/rancher/k3s/server/tls/etcd/server-ca.crt'
export ETCDCTL_CERT='/var/lib/rancher/k3s/server/tls/etcd/server-client.crt'
export ETCDCTL_KEY='/var/lib/rancher/k3s/server/tls/etcd/server-client.key'
EOF
source ~/.bashrc

# Now etcdctl commands work without specifying certs each time
sudo -E etcdctl member list --write-out=table

# Expected output:
# +------------------+---------+-----------+----------------------------+
# |        ID        | STATUS  |   NAME    |        PEER ADDRS          |
# +------------------+---------+-----------+----------------------------+
# | 1a2b3c4d5e6f7890 | started | k3-node1  | https://192.168.4.101:2380 |
# | 2b3c4d5e6f789012 | started | k3-node2  | https://192.168.4.102:2380 |
# | 3c4d5e6f78901234 | started | k3-node3  | https://192.168.4.103:2380 |
# +------------------+---------+-----------+----------------------------+

sudo -E etcdctl endpoint health
# Expected output:
# https://127.0.0.1:2379 is healthy: successfully committed proposal: took = 2.1ms
```

5. Kube-VIP — Floating Virtual IP

Kube-vip provides a floating virtual IP address for the Kubernetes API server. If the leader node goes down, the VIP automatically migrates to another control plane node. A single DaemonSet is deployed for the IPv4 VIP (192.168.4.100), announced with ARP. A second IPv6/NDP DaemonSet was tried and dropped because it was unreliable; see the note in Section 5.3.

All commands in this section are run from k3-node1 (where the KUBECONFIG export from Section 4.1 is still active). The kube-vip DaemonSets use a nodeAffinity rule that targets only nodes with the node-role.kubernetes.io/control-plane label, so Kubernetes automatically schedules a kube-vip pod on each of the three server nodes (k3-node1, k3-node2, k3-node3). Nothing is installed on the worker node. You only need to apply the manifests once from k3-node1 — the DaemonSet controller handles distributing pods to all matching nodes.

5.1 Deploy RBAC

Apply the kube-vip RBAC manifest (run once from k3-node1):

```
sudo -E kubectl apply -f https://kube-vip.io/manifests/rbac.yaml
```

5.2 Deploy IPv4 VIP DaemonSet

```
# Pinned to the version this cluster runs (October 2026). The old
# "releases | .[0].tag_name" lookup could pick up a pre-release.
export KVVERSION=v1.0.4

cat <<EOF | sudo -E kubectl apply -f -
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: kube-vip
  namespace: kube-system
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: kube-vip
  template:
    metadata:
      labels:
        app.kubernetes.io/name: kube-vip
    spec:
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
            - matchExpressions:
              - key: node-role.kubernetes.io/control-plane
                operator: Exists
      tolerations:
      - effect: NoSchedule
        operator: Exists
      - effect: NoExecute
        operator: Exists
      containers:
      - name: kube-vip
        image: ghcr.io/kube-vip/kube-vip:${KVVERSION}
        args:
        - manager
        env:
        - name: vip_arp
          value: "true"
        - name: port
          value: "6443"
```

```
        - name: vip_interface
          value: "eth0"
        - name: vip_cidr
          value: "32"
        - name: address
          value: "192.168.4.100"
        - name: cp_enable
          value: "true"
        - name: cp_namespace
          value: "kube-system"
        - name: svc_enable
          value: "false"
        - name: vip_leaderelection
          value: "true"
        securityContext:
          capabilities:
            add:
            - NET_ADMIN
            - NET_RAW
      hostNetwork: true
      serviceAccountName: kube-vip
EOF
```

5.3 Verify VIP

```
# Test IPv4 VIP
ping -c 2 192.168.4.100

# API server should respond on VIP
curl -k https://192.168.4.100:6443/version
```

Note on IPv6 VIP: Kube-vip's IPv6 NDP support is unreliable when running as a separate DaemonSet alongside the IPv4 instance — leader election conflicts prevent the IPv6 address from being acquired. The control plane VIP is IPv4-only (192.168.4.100). For IPv6 API access, there is no floating VIP equivalent. Instead, configure your kubeconfig with multiple contexts pointing to each server node's IPv6 address, as described below.

5.3.1 IPv6 kubeconfig with Multi-Node Failover

Since there is no IPv6 VIP, define all three server nodes as separate clusters in your kubeconfig. This lets you switch to a healthy node with a single command if one goes down. The TLS SANs in the K3s configuration already include all node IPv6 addresses, so certificate validation will succeed against any server node.

Create or merge the following into your `~/.kube/config` (replace the `certificate-authority-data`, `client-certificate-data`, and `client-key-data` with the values from your existing kubeconfig):

```
apiVersion: v1
kind: Config
clusters:
- name: turing-pi-v6-node1
  cluster:
    server: https://[fd00::101]:6443
    certificate-authority-data: <your-ca-data>
- name: turing-pi-v6-node2
  cluster:
    server: https://[fd00::102]:6443
    certificate-authority-data: <your-ca-data>
- name: turing-pi-v6-node3
  cluster:
    server: https://[fd00::103]:6443
    certificate-authority-data: <your-ca-data>
contexts:
- name: turing-pi-v6-node1
  context:
    cluster: turing-pi-v6-node1
    user: default
- name: turing-pi-v6-node2
  context:
    cluster: turing-pi-v6-node2
    user: default
- name: turing-pi-v6-node3
  context:
    cluster: turing-pi-v6-node3
    user: default
current-context: turing-pi-v6-node1
users:
- name: default
  user:
    client-certificate-data: <your-cert-data>
    client-key-data: <your-key-data>
```

To extract the certificate values from your existing kubeconfig on any server node:

```
sudo cat /etc/rancher/k3s/k3s.yaml
```

Copy the `certificate-authority-data`, `client-certificate-data`, and `client-key-data` values from that file into the kubeconfig above. These values are the same across all server nodes in the cluster.

5.3.2 Switching Between IPv6 Nodes

By default, kubectl will use node1 (fd00::101). If that node is down, switch to another server node:

```
# Switch to node2
kubectl config use-context turing-pi-v6-node2
# Verify which context is active
kubectl config current-context
# List all available contexts
kubectl config get-contexts
```

For day-to-day use, the IPv4 VIP (192.168.4.100) with kube-vip provides automatic failover and should remain your primary kubeconfig context. The IPv6 multi-context setup is for situations where you specifically need IPv6 connectivity or want to verify IPv6 API access.

5.4 Update Worker Node to Use VIP

Now that kube-vip is running, update k3-node4's config to point at the VIP for resilience:

```
# On k3-node4, edit the config
sudo sed -i 's|server: https://192.168.4.101:6443|server: https://192.168.4.100:6443|'
\
  /etc/rancher/k3s/config.yaml

sudo systemctl restart k3s-agent
```

5.5 Configure kubectl Access on All Nodes

Up to this point all kubectl commands have required sudo because the K3s kubeconfig at /etc/rancher/k3s/k3s.yaml is owned by root, and the worker node has no kubeconfig at all. Before proceeding, set up non-root kubectl access on every node so that the remaining sections can use kubectl without sudo.

On each server node (k3-node1, k3-node2, k3-node3):

```
mkdir -p ~/.kube
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown $(id -u):$(id -g) ~/.kube/config
chmod 600 ~/.kube/config

# Point at the VIP for HA resilience
sed -i 's/127.0.0.1/192.168.4.100/g' ~/.kube/config

# Make it permanent
echo 'export KUBECONFIG=~/.kube/config' >> ~/.bashrc
export KUBECONFIG=~/.kube/config

# Verify
kubectl get nodes
```

On the worker node (k3-node4):

```
mkdir -p ~/.kube

# Copy kubeconfig from a server node (worker has none by default)
scp ubuntu@192.168.4.101:/etc/rancher/k3s/k3s.yaml ~/.kube/config
chmod 600 ~/.kube/config

# Point at the VIP
sed -i 's/127.0.0.1/192.168.4.100/g' ~/.kube/config

# Make it permanent
echo 'export KUBECONFIG=~/.kube/config' >> ~/.bashrc
export KUBECONFIG=~/.kube/config

# Verify
kubectl get nodes
```

All kubectl commands from this point forward can be run without sudo from any node in the cluster.

6. MetalLB — Dual-Stack Load Balancer

MetalLB provides LoadBalancer-type service IPs on bare metal. In L2 mode, it uses ARP (IPv4) and NDP (IPv6) to advertise addresses on your LAN.

All commands in this section are run from any node where kubectl is configured (Section 5.5). MetalLB is deployed as Kubernetes resources — running kubectl apply submits the manifests to the cluster API, and Kubernetes schedules MetalLB's controller and speaker pods across the cluster automatically. The

controller runs on one node, while a speaker pod runs on every node (including the worker) to handle address advertisement. You do not need to install anything manually on each node.

6.1 Install MetalLB

Run from any node with kubectl access:

```
# Pin a release tag. Do not use the floating "main" branch: it is not a
# release, and this cluster had to be moved off it in April 2026.
kubectl apply -f \
  https://raw.githubusercontent.com/metallb/metallb/v0.15.3/config/manifests/metallb-
native.yaml

# Wait for the controller deployment to be ready
kubectl -n metallb-system rollout status deploy/controller

# Wait for the speaker daemonset to be ready
kubectl -n metallb-system rollout status daemonset/speaker

# Verify all MetalLB pods are running
kubectl get pods -n metallb-system
```

6.2 Configure Dual-Stack IP Pool

MetalLB requires both IPv4 and IPv6 addresses in a single pool for dual-stack allocation. If you use separate pools for each address family, services will only receive an IPv4 address. Create a combined dual-stack pool:

```
cat <<EOF | kubectl apply -f -
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: default-pool
  namespace: metallb-system
spec:
  addresses:
  - 192.168.4.200-192.168.4.220
  - fd00::200-fd00::220
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: default
  namespace: metallb-system
spec:
  ipAddressPools:
  - default-pool
EOF
```

Verify the pool was created with both address families:

```
kubectl get ipaddresspools.metallb.io -n metallb-system
# Should show one pool with both availableIPv4 and availableIPv6 > 0
kubectl get ipaddresspools.metallb.io -n metallb-system -o jsonpath='{range .items[*]}
{.metadata.name}: IPv4={.status.availableIPv4}, IPv6={.status.availableIPv6}{"n"}
{end}'
```

Services created with spec.ipFamilyPolicy: PreferDualStack or RequireDualStack will now receive both IPv4 and IPv6 external addresses.

7. Nginx Ingress Controller

Sections 7, 8, and 9 use Helm to deploy applications to the cluster. Helm is a client-side tool — it runs on your machine and talks to the Kubernetes API server, just like kubectl. You only need to install it once on a single node (or your remote workstation). All Helm and kubectl commands from this point forward should be run from the same node where Helm is installed. We install it on k3-node1 below.

7.1 Install Helm

Install Helm on k3-node1 (one-time setup):

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

7.2 Deploy Nginx Ingress with Dual-Stack

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update

cat <<'EOF' > /tmp/ingress-values.yaml
controller:
  service:
    ipFamilyPolicy: PreferDualStack
    ipFamilies:
      - IPv4
      - IPv6
  config:
    use-forwarded-headers: "true"
    enable-real-ip: "true"
EOF

helm install ingress-nginx ingress-nginx/ingress-nginx \
  --version 4.15.1 \
  --namespace ingress-nginx --create-namespace \
  -f /tmp/ingress-values.yaml
```

Verify the ingress controller gets dual-stack external IPs:

```
kubectl -n ingress-nginx get svc ingress-nginx-controller

# Verify both IPv4 and IPv6 external IPs are assigned
kubectl -n ingress-nginx get svc ingress-nginx-controller -o
jsonpath='{.status.loadBalancer.ingress[*].ip}'
# Should show both an IPv4 (e.g., 192.168.4.201) and IPv6 (e.g., fd00::200) address

# If only an IPv4 address appears, check that the MetalLB pool contains
# both address families in a single pool (see Section 6.2)
```

8. Longhorn — Distributed NVMe Storage

Longhorn provides replicated block storage across your 1TB NVMe drives. It creates persistent volumes that survive node failures by replicating data across multiple nodes.

8.1 Install Longhorn

Longhorn is configured with createDefaultDiskLabeledNodes, which means it only auto-creates storage disks on nodes that have the node.longhorn.io/create-default-disk label. This gives you explicit control over which nodes provide storage. Label all nodes before installing:

```
# Label all nodes for Longhorn disk auto-creation
kubectl label node k3-node1 node.longhorn.io/create-default-disk=true
kubectl label node k3-node2 node.longhorn.io/create-default-disk=true
kubectl label node k3-node3 node.longhorn.io/create-default-disk=true
kubectl label node k3-node4 node.longhorn.io/create-default-disk=true

# Verify labels
kubectl get nodes --show-labels | grep longhorn
```

Now install Longhorn:

```
helm repo add longhorn https://charts.longhorn.io
helm repo update

# Make sure the multipathd blacklist from Section 3.7 is in place on every node first.
# nodeDownPodDeletionPolicy lets StatefulSet/Deployment pods on a dead node be
# force-deleted, so their Longhorn volumes can re-attach elsewhere (live setting).
helm install longhorn longhorn/longhorn \
  --version 1.11.1 \
  --namespace longhorn-system --create-namespace \
  --set defaultSettings.defaultReplicaCount=3 \
  --set defaultSettings.createDefaultDiskLabeledNodes=true \
  --set defaultSettings.nodeDownPodDeletionPolicy=delete-both-statefulset-and-
deployment-pod

# Longhorn deploys many components — wait for everything to be ready
# This can take several minutes on first install
echo "Waiting for Longhorn pods to start..."
kubectl -n longhorn-system rollout status deploy/longhorn-ui --timeout=300s
kubectl -n longhorn-system rollout status deploy/longhorn-driver-deployer --
timeout=300s

# Verify all pods are running (no Pending or CrashLoopBackOff)
kubectl -n longhorn-system get pods
```

Once all pods are Running, verify that Longhorn detected a disk on each node:

```
kubectl -n longhorn-system get nodes.longhorn.io -o json \
  | jq '.items[] | {name: .metadata.name, disks: .spec.disks}'

# Each node should show a disk entry with:
#   "path": "/var/lib/longhorn/"
#   "allowScheduling": true
# If any node shows "disks": {}, the label was not applied before
# Longhorn started. Apply the label and wait 30 seconds for Longhorn
# to detect it.
```

Longhorn uses /var/lib/longhorn/ on each node's NVMe root filesystem and reserves approximately 30% of the disk by default. With 1TB NVMe drives, this provides roughly 700GB of usable storage per node.

8.2 Set as Default StorageClass

```
# Remove default from local-path
kubectl patch storageclass local-path -p \
  '{"metadata": {"annotations":{"storageclass.kubernetes.io/is-default-
class":"false"}}}'

# Verify Longhorn is default
kubectl get storageclass
```

8.3 Access Longhorn UI

Longhorn includes a web UI. Before running port-forward, ensure all Longhorn pods are fully running (verified in Section 8.1). If the longhorn-frontend pod is still in Pending state, the port-forward will fail with 'pod is not running'. Expose it temporarily:

```
# Confirm the frontend pod is Running first
kubectl -n longhorn-system get pods -l app=longhorn-ui

# Then forward the port
kubectl -n longhorn-system port-forward svc/longhorn-frontend 8080:80 --address
0.0.0.0
```

Access at http://\<any-node-ip>:8080. For permanent access, create an Ingress resource (shown in Section 10).

9. Management Dashboard

Choose one of the following dashboard options. Rancher is a full cluster management platform; Portainer is lighter weight and more beginner-friendly.

9.1 Option A: Rancher

Install cert-manager (required)

```
kubectl apply -f \
  https://github.com/cert-manager/cert-manager/releases/download/v1.19.3/cert-
manager.yaml

kubectl -n cert-manager rollout status deploy/cert-manager
kubectl -n cert-manager rollout status deploy/cert-manager-webhook
kubectl -n cert-manager rollout status deploy/cert-manager-cainjector
```

Install Rancher

```
helm repo add rancher-stable https://releases.rancher.com/server-charts/stable
helm repo update

kubectl create namespace cattle-system

cat <<'EOF' > /tmp/rancher-values.yaml
hostname: rancher.local
replicas: 1
bootstrapPassword: admin
ingress:
  extraAnnotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
EOF

helm install rancher rancher-stable/rancher \
  --namespace cattle-system \
  -f /tmp/rancher-values.yaml
```

Point rancher.local to your ingress controller's LoadBalancer IP in your DNS or /etc/hosts, then access https://rancher.local. You will be prompted to set a permanent password on first login.

9.2 Option B: Portainer

```
helm repo add portainer https://portainer.github.io/k8s/
helm repo update

helm install portainer portainer/portainer \
  --version 239.1.0 \
  --namespace portainer --create-namespace \
  --set service.type=LoadBalancer \
  --set service.ipFamilyPolicy=PreferDualStack
```

Current Portainer charts accept `service.ipFamilyPolicy`, which is how this cluster's release is configured. Older charts did not, and the service came up SingleStack (IPv4 only). If you are on such a chart, patch the service after installation:

```
# Only needed if the chart ignored service.ipFamilyPolicy
kubectl -n portainer patch svc portainer \
  -p '{"spec":{"ipFamilyPolicy":"PreferDualStack","ipFamilies":["IPv4","IPv6"]}}'

# Verify both IPs are assigned
kubectl -n portainer get svc portainer \
  -o jsonpath='IPv4/IPv6: {.status.loadBalancer.ingress[*].ip}{"\n"}'
```

Portainer creates a PersistentVolumeClaim for its data. If Longhorn is still initializing or the PVC is not yet bound, the Portainer pod will remain in Pending state. Check the deployment and PVC status:

```
# Watch the pod status — wait until it shows Running
kubectl -n portainer get pods -w

# If the pod is stuck in Pending, check the PVC
kubectl -n portainer get pvc
# STATUS should be "Bound". If it shows "Pending", Longhorn may
# still be provisioning. Describe the PVC for details:
kubectl -n portainer describe pvc

# Wait for the deployment to be ready
kubectl -n portainer rollout status deploy/portainer --timeout=300s

# Watch for the LoadBalancer IP to be assigned by MetalLB
kubectl -n portainer get svc portainer -w
# Wait until EXTERNAL-IP changes from <pending> to an IP address
```

Once the EXTERNAL-IP is assigned and the pod is Running, access Portainer at https://\<EXTERNAL-IP>:9443. The HTTPS certificate is self-signed, so your browser will show a security warning — this is expected. You must create an admin account within a few minutes of first startup, or Portainer will time out and require a pod restart.

Resetting the Portainer Admin Password

Warning: Portainer has no built-in password reset command. The only way to reset the admin password is to delete the Portainer PVC and recreate it, which erases all Portainer configuration including environments, users, stacks, and settings. After the reset, you must recreate the admin account and re-add your Kubernetes environment.

```
# Scale down Portainer
kubectl -n portainer scale deploy/portainer --replicas=0

# Wait for the pod to terminate
kubectl -n portainer get pods -w
# Ctrl+C once no pods remain

# Delete the PVC (THIS ERASES ALL PORTAINER DATA)
kubectl -n portainer delete pvc portainer

# Recreate the PVC
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: portainer
  namespace: portainer
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: longhorn
  resources:
    requests:
      storage: 10Gi
EOF

# Scale Portainer back up
kubectl -n portainer scale deploy/portainer --replicas=1

# Watch for the pod to be ready
kubectl -n portainer get pods -w
```

Once the pod is Running, immediately go to https://\<EXTERNAL-IP>:9443 and create a new admin account before the timeout expires. Portainer will auto-detect the local Kubernetes cluster.

10. Remote Kubeconfig Setup

Kubectl access on all cluster nodes was configured in Section 5.5. This section covers setting up kubectl on a workstation outside the cluster for remote management.

10.1 Configure kubectl on Remote Workstation

To manage the cluster from a workstation outside the cluster:

```
# Copy kubeconfig from any server node
mkdir -p ~/.kube
scp ubuntu@192.168.4.101:/etc/rancher/k3s/k3s.yaml ~/.kube/config

# Point at the VIP instead of localhost
# Linux:
sed -i 's/127.0.0.1/192.168.4.100/g' ~/.kube/config
# macOS (BSD sed requires '' after -i):
# sed -i '' 's/127.0.0.1/192.168.4.100/g' ~/.kube/config

chmod 600 ~/.kube/config

# Verify
kubectl get nodes
```

For IPv6 API access, there is no floating VIP — use the multi-context kubeconfig approach described in Section 5.3.1.

11. Cluster Validation

11.1 Verify All Nodes

```
kubectl get nodes -o wide
```

All four nodes should show status Ready.

11.2 Verify Dual-Stack Pod Networking

```
# Deploy a test pod
kubectl run test-ds --image=busybox --restart=Never -- sleep 3600

# Check dual-stack pod IPs
kubectl get pod test-ds -o jsonpath='{.status.podIPs}' | jq .
# Should return both a 10.42.x.x and fd10:42::x address

# Test connectivity from the pod
kubectl exec test-ds -- ping -c 2 192.168.4.101
kubectl exec test-ds -- ping6 -c 2 fd00::101

# Test DNS (both A and AAAA records)
kubectl exec test-ds -- nslookup kubernetes.default.svc.cluster.local

# Cleanup
kubectl delete pod test-ds
```

11.3 Verify HA Failover

```
# From your workstation, run a continuous check
watch kubectl get nodes

# On k3-node1, stop K3s to simulate a failure
sudo systemctl stop k3s

# The cluster should remain accessible via the VIP
# kube-vip migrates the VIP to k3-node2 or k3-node3
# etcd maintains quorum with 2 of 3 members
kubectl get nodes   # Should still work

# Bring k3-node1 back
sudo systemctl start k3s
```

11.4 Verify Services Get Dual-Stack IPs

```
# Check all LoadBalancer services for dual-stack IPs
kubectl get svc -A -o wide | grep LoadBalancer

# Verify ingress-nginx has both IPv4 and IPv6 external IPs
kubectl -n ingress-nginx get svc ingress-nginx-controller \
  -o jsonpath='IPv4/IPv6: {.status.loadBalancer.ingress[*].ip}{"\n"}'

# Verify Portainer has both IPv4 and IPv6 external IPs
kubectl -n portainer get svc portainer \
  -o jsonpath='IPv4/IPv6: {.status.loadBalancer.ingress[*].ip}{"\n"}'

# Test connectivity on both stacks
INGRESS_V4=$(kubectl -n ingress-nginx get svc ingress-nginx-controller \
  -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
INGRESS_V6=$(kubectl -n ingress-nginx get svc ingress-nginx-controller \
  -o jsonpath='{.status.loadBalancer.ingress[1].ip}')

curl -sk https://${INGRESS_V4} -o /dev/null -w "IPv4: %{http_code}\n"
curl -sk https://[${INGRESS_V6}] -o /dev/null -w "IPv6: %{http_code}\n"
# Both should return a status code (404 is normal with no ingress rules defined)
```

Each LoadBalancer service should show two external IPs: one from the 192.168.4.200-220 range and one from the fd00::200-220 range. If services only show an IPv4 address, verify that the MetalLB pool in Section 6.2 contains both address families in a single IPAddressPool. Services created before a correct dual-stack pool was configured will need to be restarted to pick up the IPv6 address.

12. Optional: Prometheus + Grafana Monitoring

The kube-prometheus-stack Helm chart deploys Prometheus (metrics collection), Grafana (visualization), and a set of recording/alerting rules tuned for Kubernetes. It includes Node Exporter on every node for hardware-level metrics and pre-built dashboards for cluster, node, and workload monitoring.

12.1 Install kube-prometheus-stack

```
helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts
helm repo update

cat <<'EOF' > /tmp/monitoring-values.yaml
grafana:
  service:
    type: LoadBalancer
    ipFamilyPolicy: PreferDualStack
  adminPassword: admin
prometheus:
  prometheusSpec:
    storageSpec:
      volumeClaimTemplate:
        spec:
          # local-path rather than longhorn, as on this cluster: Prometheus data
          # is rebuildable, so it doesn't need 3 replicas over the 1 GbE backplane
          storageClassName: local-path
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 20Gi
    # Scrape all ServiceMonitors across all namespaces
    serviceMonitorSelectorNilUsesHelmValues: false
    podMonitorSelectorNilUsesHelmValues: false
    # Retention period for metrics
    retention: 15d
EOF

helm install monitoring prometheus-community/kube-prometheus-stack \
  --version 83.6.0 \
  --namespace monitoring --create-namespace \
  -f /tmp/monitoring-values.yaml
```

Cluster as built (October 2026): Grafana is on 192.168.4.202. Prometheus itself is also exposed as a dual-stack LoadBalancer on 192.168.4.210 / fd00::210 (`prometheus.service` with a `metallb.io/loadBalancerIPs` annotation). Grafana login goes through Zitadel OIDC (`grafana.ini` `auth.generic_oauth`); see the `zitadel/` and `grafana/` folders. The `admin`/`admin` password above is only a first-install placeholder.

Wait for all monitoring pods to be ready:

```
kubectl -n monitoring rollout status deploy/monitoring-grafana --timeout=300s
kubectl -n monitoring rollout status deploy/monitoring-kube-prometheus-stack-operator
--timeout=300s

# Verify all pods are running
kubectl -n monitoring get pods
```

12.2 Access Grafana

```
# Get the Grafana LoadBalancer IPs
kubectl -n monitoring get svc monitoring-grafana

# Patch for dual-stack if only IPv4 was assigned
kubectl -n monitoring patch svc monitoring-grafana \
  -p '{"spec":{"ipFamilyPolicy":"PreferDualStack","ipFamilies":["IPv4","IPv6"]}}'

# Verify dual-stack
kubectl -n monitoring get svc monitoring-grafana \
  -o jsonpath='Grafana: {.status.loadBalancer.ingress[*].ip}{"\n"}'
```

Open http://\<EXTERNAL-IP> in your browser. Log in with username 'admin' and password 'admin' (as set in the values file). Change the password on first login. If you forget the password or need to reset it:

```
kubectl -n monitoring exec deploy/monitoring-grafana -c grafana -- grafana cli admin
reset-admin-password <new-password>
```

12.3 Recommended Dashboards

The kube-prometheus-stack includes many pre-built dashboards. The most useful ones for monitoring a K3s cluster on Turing Pi hardware are listed below. All built-in dashboards are available immediately under the 'Default' folder in Grafana. The community dashboards configured in the values file above are under the 'Community' folder.

Built-in Dashboards (already included)

Kubernetes / Compute Resources / Cluster — overall CPU, memory, and network usage across all nodes. This is the best starting point for a cluster health overview.

Kubernetes / Compute Resources / Node (Pods) — per-node breakdown showing which pods consume CPU and memory. Useful for identifying resource-hungry workloads on specific RK1 nodes.

Node Exporter / Nodes — bare-metal hardware metrics including CPU utilization per core, memory pressure, disk I/O throughput, and filesystem usage. Critical for monitoring NVMe health and the thermal performance of the RK3588 SoCs.

etcd — etcd cluster health, leader elections, proposal commit latency, and database size. Important for monitoring the HA control plane and catching etcd performance issues early.

CoreDNS — DNS query rates, error rates, and cache hit ratios. Useful for diagnosing service discovery issues in the cluster.

Kubernetes / Networking / Cluster — network bandwidth by namespace, pod-to-pod traffic rates, and packet drop rates. Helps identify networking bottlenecks on the Turing Pi 2.5 backplane.

Community Dashboards (import manually)

These dashboards are available from the Grafana community library and must be imported manually. To import a dashboard: go to Dashboards > New > Import, enter the dashboard ID, click Load, select 'Prometheus' as the data source, then click Import.

Node Exporter Full (ID: 1860) — the most comprehensive hardware dashboard available. Shows per-core CPU frequency, temperatures, detailed disk latency histograms, network interface statistics, and memory breakdown. Especially valuable for the RK1 since the RK3588 has big.LITTLE cores (4x A76 + 4x A55) and you can monitor each core individually.

Kubernetes / Views / Global (ID: 15759) — a clean, high-level cluster overview with namespace-level resource consumption, pod counts, and deployment status. Good as a Grafana home dashboard.

12.4 Add Longhorn Monitoring

Longhorn can expose storage metrics to Prometheus via a ServiceMonitor. Since we set serviceMonitorSelectorNilUsesHelmValues to false, Prometheus will automatically discover any ServiceMonitor in any namespace. Enable Longhorn metrics:

```
# Create a ServiceMonitor for Longhorn
cat <<EOF | kubectl apply -f -
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: longhorn-prometheus
  namespace: longhorn-system
  labels:
    name: longhorn-prometheus
spec:
  selector:
    matchLabels:
      app: longhorn-manager
  namespaceSelector:
    matchNames:
    - longhorn-system
  endpoints:
  - port: manager
EOF

# Verify Prometheus is scraping Longhorn (may take a minute)
kubectl -n monitoring port-forward svc/monitoring-kube-prometheus-prometheus 9090:9090
--address 0.0.0.0 &
sleep 5
curl -s http://localhost:9090/api/v1/targets | jq '.data.activeTargets[] |
select(.labels.job=="longhorn-system/longhorn-prometheus") | .health'
kill %1 2>/dev/null
```

Once Longhorn metrics are flowing, import the Longhorn dashboard in Grafana using the same import method: Dashboards > New > Import, enter ID 13032, select the Prometheus data source, and click Import.

The Longhorn dashboard (ID: 13032) shows volume health status, replica counts, read/write IOPS per volume, throughput, and latency. It also displays node storage capacity and usage, making it easy to monitor your 1TB NVMe drives across all four nodes.

12.5 Set a Home Dashboard

To set a default home dashboard that appears when you open Grafana:

```
# In the Grafana UI:
# 1. Navigate to the dashboard you want as your home page
#    (recommended: "Kubernetes / Compute Resources / Cluster"
#     or "Kubernetes / Views / Global")
# 2. Click the star icon to mark it as a favorite
# 3. Go to Administration > General > Default preferences
# 4. Under "Home Dashboard", select your chosen dashboard
# 5. Click Save
```

13. Maintenance and Management

13.1 Useful Commands

```
# Check cluster status
kubectl get nodes -o wide
kubectl get pods -A

# Check etcd health (requires etcdctl installed per Section 4.6)
# Or use: kubectl get --raw=/healthz/etcd
sudo -E etcdctl endpoint health

# Manual etcd snapshot
sudo k3s etcd-snapshot save

# List etcd snapshots
sudo k3s etcd-snapshot ls

# K3s service management
sudo systemctl status k3s         # Server nodes
sudo systemctl status k3s-agent   # Worker node
sudo systemctl restart k3s
sudo journalctl -u k3s -f         # Follow logs
```

13.2 Taking a Single Node Out of Service

This procedure safely removes a node from the cluster for hardware repairs, OS upgrades, or reboots. The process differs slightly between server (control plane) and worker nodes.

Worker Node (k3-node4)

Worker nodes are the simplest to maintain since they do not run etcd or control plane components.

```
# Step 1: Cordon the node (prevents new pods from being scheduled)
kubectl cordon k3-node4

# Step 2: Drain the node (evicts running pods to other nodes)
kubectl drain k3-node4 --ignore-daemonsets --delete-emptydir-data --timeout=120s

# Step 3: Verify pods have been evicted
kubectl get pods -A -o wide | grep k3-node4
# Only DaemonSet pods (longhorn, node-exporter, speaker) should remain
# (kube-vip runs only on the control-plane nodes)

# Step 4: Perform maintenance on k3-node4
# SSH to k3-node4 and do your work:
sudo apt update && sudo apt upgrade -y
sudo reboot

# Step 5: Wait for the node to come back online
# From any other node, watch for it:
kubectl get nodes -w
# Wait until k3-node4 shows Ready (may take 1-2 minutes after reboot)

# Step 6: Uncordon the node (allow pods to be scheduled again)
kubectl uncordon k3-node4

# Step 7: Verify node is healthy
kubectl get nodes
kubectl get pods -A -o wide | grep k3-node4
```

Longhorn warning: When a node is drained, Longhorn will rebuild volume replicas on the remaining nodes. This is normal. When the node returns, Longhorn will rebalance replicas back to it. If the node is offline for more than 30 minutes, Longhorn may start rebuilding all replicas permanently. For short maintenance windows, this is harmless but generates I/O. You can monitor replica health in the Longhorn UI or with: kubectl -n longhorn-system get replicas.longhorn.io

Troubleshooting — Longhorn webhook blocks uncordon: If kubectl uncordon fails with 'failed calling webhook validator.longhorn.io: no endpoints available', Longhorn's admission webhook is down. This blocks any node state changes. Temporarily delete the webhook, uncordon the node, then restart Longhorn to recreate it:

```
# Delete the blocking webhook
kubectl delete validatingwebhookconfiguration longhorn-webhook-validator

# Uncordon the node
kubectl uncordon k3-node4

# Restart Longhorn components to recreate the webhook
kubectl -n longhorn-system rollout restart deploy/longhorn-driver-deployer
kubectl -n longhorn-system rollout restart deploy/longhorn-ui
kubectl -n longhorn-system rollout restart daemonset/longhorn-manager

# Verify the webhook is recreated
kubectl get validatingwebhookconfiguration | grep longhorn
```

Server Node (k3-node1, k3-node2, or k3-node3)

Critical: Never take more than one server node offline at the same time. etcd requires a majority quorum (2 of 3 nodes) to function. If two server nodes go down simultaneously, the cluster loses quorum and becomes read-only — no pods can be scheduled, no resources can be created or modified.

```
# Step 1: Verify etcd health before starting
kubectl get --raw=/healthz/etcd
# Must return "ok" before proceeding

# Step 2: Take a manual etcd snapshot as a safety measure
# Run on the node you are NOT taking offline:
sudo k3s etcd-snapshot save

# Step 3: Cordon and drain the target server node
kubectl cordon k3-node2
kubectl drain k3-node2 --ignore-daemonsets --delete-emptydir-data --timeout=120s

# Step 4: Verify the VIP has migrated (if you are draining the VIP holder)
ping -c 2 192.168.4.100
# If the VIP was on this node, kube-vip automatically moves it

# Step 5: Perform maintenance on k3-node2
sudo apt update && sudo apt upgrade -y
sudo reboot

# Step 6: Wait for the node to rejoin
kubectl get nodes -w
# Wait until k3-node2 shows Ready

# Step 7: Verify etcd health after the node rejoins
kubectl get --raw=/healthz/etcd

# Step 8: Uncordon the node
kubectl uncordon k3-node2
```

Warning on K3s upgrades: When upgrading K3s on server nodes, always upgrade one node at a time. Wait for each node to fully rejoin the cluster and for etcd to report healthy before upgrading the next. Upgrading multiple server nodes simultaneously can cause etcd quorum loss and cluster downtime. The system-upgrade-controller Plans in Section 13.5 enforce this with `concurrency: 1`.

If you need to take k3-node1 specifically offline and it holds the VIP, all kubectl commands must be run from another server node (k3-node2 or k3-node3) where kubectl is configured. The VIP will migrate automatically, but your active SSH session to k3-node1 will not.

13.3 Graceful Full Cluster Shutdown

Follow this procedure for a complete physical power-down of the Turing Pi board, such as for relocation, hardware upgrades, or extended downtime. The shutdown order matters: worker first, then server nodes

in reverse join order, with the initial server node (k3-node1) last.

Warning: A full shutdown stops all workloads. Any in-flight requests will be dropped. Stateful applications should be stopped gracefully before draining. Longhorn volumes will be unavailable during the shutdown — this is expected.

Step 1: Stop workloads gracefully (optional)

```
# Scale down your own application deployments that write data
# kubectl -n <namespace> scale deploy/<n> --replicas=0

# Wait for pods to terminate
kubectl get pods -A | grep Terminating
```

Do NOT cordon nodes before shutdown: Cordoning sets SchedulingDisabled on nodes, and this state persists across reboots. When the cluster restarts with all nodes cordoned, no pods can be scheduled. CoreDNS fails to start, which cascades into a cluster-wide DNS failure. Without DNS, Longhorn and other controllers cannot communicate with their backends and may scale their deployments to 0 replicas. This creates a difficult recovery scenario requiring manual intervention on every component. Simply stopping K3s services directly (next steps) is the correct approach for a clean shutdown.

Step 2: Stop the worker node first

```
# On k3-node4:
sudo systemctl stop k3s-agent
```

Step 3: Stop server nodes (reverse join order)

```
# On k3-node3 (last server joined):
sudo systemctl stop k3s

# On k3-node2 (second server joined):
sudo systemctl stop k3s

# On k3-node1 (initial server — ALWAYS stop last):
sudo systemctl stop k3s
```

Important: k3-node1 must be the last server node stopped and the first server node started. As the initial cluster node, it is the safest leader for etcd recovery. Stopping it while other servers are still running and then starting it last during recovery can lead to etcd inconsistencies.

Step 4: Power down

```
# On each node (or via BMC):
sudo shutdown -h now

# Or power off the entire Turing Pi board via BMC once all nodes
# show as powered off or have completed their shutdown
```

13.4 Restarting the Cluster After Power-Down

When the Turing Pi board is powered on, each RK1 node boots automatically. K3s is installed as a systemd service with automatic start enabled, so the cluster will attempt to self-recover. However, the startup order matters for a clean recovery.

Startup Order

The ideal startup order is the reverse of the shutdown order:

1. k3-node1 (initial server) — must start first. It bootstraps the etcd cluster and becomes the initial leader.
2. k3-node2 and k3-node3 (server nodes) — start after k3-node1 is running. They rejoin the etcd cluster.
3. k3-node4 (worker) — start last. It connects to the API server via the VIP.

Automatic Recovery (all nodes power on simultaneously)

If all nodes power on at the same time (typical when the Turing Pi board receives power), K3s will still recover, but it may take longer. The etcd cluster needs a majority of members to be online before it elects a leader. With three server nodes, etcd requires two nodes to be running before it becomes functional. The sequence is:

1. All nodes boot and K3s services start automatically (systemd: enabled).
2. etcd on the server nodes attempts to form a cluster. It may take 1–3 minutes for leader election to complete.
3. Once etcd has quorum, the Kubernetes API server becomes available.
4. kube-vip acquires the VIP on one of the server nodes.
5. The worker node (k3-node4) connects to the API server and begins running pods.
6. Longhorn rebuilds any degraded volume replicas.

Post-Recovery Verification Checklist

After the cluster powers on, run through this checklist from any server node. Wait 2–5 minutes after power-on before starting.

```
# === STEP 1: Check node status ===
kubectl get nodes
# All 4 nodes should show Ready. If any show NotReady, check K3s
# service on that node: ssh <node> 'sudo journalctl -u k3s -n 30'

# === STEP 2: Uncordon any SchedulingDisabled nodes ===
kubectl get nodes | grep SchedulingDisabled
# If any nodes show SchedulingDisabled, uncordon them:
# kubectl uncordon <node-name>
# If uncordon fails with a Longhorn webhook error, delete the webhook:
# kubectl delete validatingwebhookconfiguration longhorn-webhook-validator
# Then retry the uncordon

# === STEP 3: Check etcd health ===
kubectl get --raw=/healthz/etcd
# Must return "ok"

# === STEP 4: Verify the VIP is responding ===
ping -c 2 192.168.4.100

# === STEP 5: Verify CoreDNS is running ===
kubectl -n kube-system get deploy coredns
# READY should show 1/1. If it shows 0/0:
# kubectl -n kube-system scale deploy/coredns --replicas=1

# === STEP 6: Check for deployments scaled to 0 ===
# After a disruptive restart, some deployments may be at 0 replicas.
# This check catches the problem before it cascades.
echo "--- Deployments at 0/0 replicas ---"
kubectl get deploy -A | awk '\$3 == "0/0" {print}'
# If any show 0/0, scale them back up. Common ones:
# kubectl -n kube-system scale deploy metrics-server --replicas=1
# kubectl -n kube-system scale deploy local-path-provisioner --replicas=1
# kubectl -n metallb-system scale deploy controller --replicas=1
# kubectl -n cert-manager scale deploy cert-manager --replicas=1
# kubectl -n cert-manager scale deploy cert-manager-cainjector --replicas=1
# kubectl -n cert-manager scale deploy cert-manager-webhook --replicas=1
# kubectl -n ingress-nginx scale deploy ingress-nginx-controller --replicas=1
# kubectl -n longhorn-system scale deploy csi-attacher --replicas=3
# kubectl -n longhorn-system scale deploy csi-provisioner --replicas=3
# kubectl -n longhorn-system scale deploy csi-resizer --replicas=3
# kubectl -n longhorn-system scale deploy csi-snapshotter --replicas=3
```

```
# kubectl -n longhorn-system scale deploy longhorn-driver-deployer --replicas=1
# kubectl -n longhorn-system scale deploy longhorn-ui --replicas=2
# kubectl -n monitoring scale deploy monitoring-grafana --replicas=1
# kubectl -n monitoring scale deploy monitoring-kube-prometheus-operator --replicas=1
# kubectl -n monitoring scale deploy monitoring-kube-state-metrics --replicas=1
# kubectl -n portainer scale deploy portainer --replicas=1

# === STEP 7: Verify Longhorn health ===
kubectl -n longhorn-system get pods | grep -v Running | grep -v Completed
# If Longhorn manager pods are in CrashLoopBackOff with webhook errors:
# kubectl delete validatingwebhookconfiguration longhorn-webhook-validator
# kubectl delete mutatingwebhookconfiguration longhorn-webhook-mutator
# kubectl -n longhorn-system delete pods -l app=longhorn-manager

# Check volume health
kubectl -n longhorn-system get volumes.longhorn.io
# Volumes should show "attached" with "healthy" robustness
# "degraded" is normal temporarily while replicas resync

# === STEP 8: Verify all pods are running ===
kubectl get pods -A | grep -v Running | grep -v Completed
# Some pods may show 1-2 restarts — this is normal after a power cycle
# Look for Pending, CrashLoopBackOff, or Error states

# === STEP 9: Verify LoadBalancer services have dual-stack IPs ===
kubectl get svc -A | grep LoadBalancer
```

Warning — etcd recovery failure: If the cluster does not recover after 5–10 minutes, check if etcd has quorum. Run 'sudo journalctl -u k3s | grep etcd' on each server node and look for 'etcdserver: publish error' or 'rafthttp: failed to reach peer' messages. If etcd cannot form quorum (e.g., because a server node failed to boot), you may need to restore from an etcd snapshot — see Section 13.6.

Warning — Longhorn volume degradation: After a full power cycle, Longhorn volumes may temporarily show as Degraded while replicas resync. This is normal and resolves automatically within minutes, depending on volume sizes. Do not delete or recreate volumes during this period. Monitor progress in the Longhorn UI or with: kubectl -n longhorn-system get replicas.longhorn.io

Warning — data corruption risk: The primary corruption risk occurs during an unclean shutdown (sudden power loss without the graceful procedure above). etcd is designed to handle this through its write-ahead log, but rapid repeated power cycles can corrupt the etcd database. Longhorn volumes are also at risk during unclean shutdowns if writes were in progress. Always use the graceful shutdown procedure when possible. If the cluster was shut down uncleanly, check etcd and Longhorn health immediately after recovery.

13.5 K3s Upgrades

Always upgrade one node at a time. The order is: server nodes first (one by one), then the worker node.

Primary method: system-upgrade-controller (SUC). This cluster upgrades k3s with Rancher's system-upgrade-controller (v0.19.0, namespace `system-upgrade`) and two Plans:

Plan	Targets	Behaviour
`k3s-server-upgrade`	nodes with `node-role.kubernetes.io/control-plane`	`concurrency: 1`, `cordon: true`
`k3s-agent-upgrade`	nodes without the control-plane role (k3-node4)	`concurrency: 1`, `cordon: true`; its `prepare` step waits for `k3s-server-upgrade` to finish

Both Plans use the `rancher/k3s-upgrade` image. They run at v1.37.1+k3s1 as of October 2026. To upgrade, take a snapshot, then set the same new version in **both** Plans:

```
# Step 1: Check current version
kubectl get nodes -o wide
kubectl -n system-upgrade get plans -o custom-
columns=NAME:.metadata.name,VERSION:.spec.version

# Step 2: Take an etcd snapshot before upgrading (on a server node)
sudo k3s etcd-snapshot save

# Step 3: Set the target version in both Plans (edit .spec.version)
kubectl edit plan k3s-server-upgrade -n system-upgrade
kubectl edit plan k3s-agent-upgrade -n system-upgrade

# Step 4: Watch the controller upgrade servers one by one, then the agent
kubectl -n system-upgrade get jobs,pods -w
kubectl get nodes -w
kubectl get --raw=/healthz/etcd
```

To install SUC on a fresh cluster, apply the release manifests from github.com/rancher/system-upgrade-controller (pin the release, e.g. v0.19.0), then create the two Plans described above.

Manual fallback (no SUC). Always pin the target version. Without `INSTALL_K3S_VERSION`, get.k3s.io installs the current stable release, which may not be the version you meant to deploy.

```
# Set to the release you are upgrading to
export K3S_TARGET=v1.37.1+k3s1

# Step 1: Check current version
kubectl get nodes -o wide

# Step 2: Take an etcd snapshot before upgrading
sudo k3s etcd-snapshot save

# Step 3: Upgrade server nodes one at a time
# On k3-node1:
kubectl drain k3-node1 --ignore-daemonsets --delete-emptydir-data
curl -sfL https://get.k3s.io | INSTALL_K3S_VERSION=${K3S_TARGET} sh -
kubectl uncordon k3-node1
kubectl get nodes    # Wait for k3-node1 to show Ready with new version
kubectl get --raw=/healthz/etcd   # Verify etcd healthy before next node

# Repeat for k3-node2, then k3-node3

# Step 4: Upgrade the worker node
# On k3-node4:
kubectl drain k3-node4 --ignore-daemonsets --delete-emptydir-data
curl -sfL https://get.k3s.io | INSTALL_K3S_VERSION=${K3S_TARGET}
K3S_URL=https://192.168.4.100:6443 \
  K3S_TOKEN="<TOKEN>" sh -
kubectl uncordon k3-node4

# Step 5: Verify all nodes are on the new version
kubectl get nodes -o wide
```

If you upgrade by hand while the SUC Plans exist, set the Plans to the same version afterwards. Otherwise the controller will move the nodes back to the Plan version.

13.6 Backups

A comprehensive backup strategy for K3s covers three layers: etcd snapshots (cluster state), Longhorn volume backups (persistent data), and Kubernetes resource manifests (configuration). Each layer protects against different failure modes.

etcd Snapshots (Cluster State)

etcd stores all Kubernetes cluster state: deployments, services, secrets, configmaps, and RBAC. An etcd snapshot lets you restore the entire cluster to a previous state. K3s is already configured (in Section 4.1) to take automated snapshots every 6 hours with 10 retained.

```
# View automated snapshot schedule (configured in /etc/rancher/k3s/config.yaml)
# etcd-snapshot-schedule-cron: "0 */6 * * *"
# etcd-snapshot-retention: 10

# List existing snapshots
sudo k3s etcd-snapshot ls

# Take a manual snapshot (recommended before upgrades or changes)
sudo k3s etcd-snapshot save

# Snapshots are stored at:
ls /var/lib/rancher/k3s/server/db/snapshots/

# Copy snapshots off-node for safety (run from your workstation)
scp ubuntu@192.168.4.101:/var/lib/rancher/k3s/server/db/snapshots/* \
  ~/k3s-backups/etcd/
```

Restoring from an etcd Snapshot

Warning: Restoring an etcd snapshot resets the entire cluster state to the snapshot point in time. Any changes made after the snapshot was taken will be lost. This is a last-resort recovery procedure.

```
# Step 1: Stop K3s on ALL server nodes
# On k3-node3: sudo systemctl stop k3s
# On k3-node2: sudo systemctl stop k3s
# On k3-node1: sudo systemctl stop k3s

# Step 2: Restore on k3-node1 (initial server node ONLY)
sudo k3s server \
  --cluster-reset \
  --cluster-reset-restore-path=/var/lib/rancher/k3s/server/db/snapshots/<snapshot-
name>

# Wait for the message:
# "Managed etcd cluster membership has been reset, restart without
#  --cluster-reset flag now."

# Step 3: Start K3s normally on k3-node1
sudo systemctl start k3s

# Step 4: On k3-node2 and k3-node3, delete old etcd data and rejoin
# On each secondary server:
sudo systemctl stop k3s
sudo rm -rf /var/lib/rancher/k3s/server/db/etcd
sudo systemctl start k3s

# Step 5: Verify recovery
kubectl get nodes
kubectl get --raw=/healthz/etcd
```

Longhorn Volume Backups (Persistent Data)

etcd snapshots do not include the actual data stored in Longhorn volumes (databases, application files, etc.). Longhorn provides its own backup mechanism that can write volume snapshots to an external S3-compatible store or NFS share.

Longhorn Snapshots (local, fast)

Longhorn snapshots are point-in-time copies stored on the same NVMe drives. They are fast to create and useful for quick rollbacks, but do not protect against disk failure.

```
# Create a snapshot of all volumes via the Longhorn UI:
# 1. Open the Longhorn UI (kubectl -n longhorn-system port-forward
#    svc/longhorn-frontend 8080:80 --address 0.0.0.0)
# 2. Go to Volume > select a volume > click "Take Snapshot"

# Or create snapshots via kubectl:
kubectl -n longhorn-system get volumes.longhorn.io

# Create a snapshot for a specific volume
cat <<EOF | kubectl apply -f -
apiVersion: longhorn.io/v1beta2
kind: Snapshot
metadata:
  name: manual-backup-$(date +%Y%m%d-%H%M%S)
  namespace: longhorn-system
spec:
  volume: <volume-name>
EOF
```

Longhorn Backups to External Storage (off-cluster)

For true disaster recovery, configure Longhorn to write backups to an external S3-compatible store (such as MinIO, AWS S3, or Backblaze B2) or an NFS share. This protects against total cluster loss.

```
# Configure backup target in Longhorn UI:
# 1. Go to Setting > General > Backup Target
# 2. Set the target URL:
#    S3:  s3://bucket-name@region/folder
#    NFS: nfs://server-ip:/path/to/share
# 3. For S3, set the Backup Target Credential Secret:

# Create the S3 credentials secret
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Secret
metadata:
  name: longhorn-backup-s3
  namespace: longhorn-system
type: Opaque
stringData:
  AWS_ACCESS_KEY_ID: <your-access-key>
  AWS_SECRET_ACCESS_KEY: <your-secret-key>
  AWS_ENDPOINTS: <s3-endpoint-url>
EOF

# 4. Set "Backup Target Credential Secret" to "longhorn-backup-s3"
# 5. Test the connection with "Test Connection" button
```

Once configured, you can create backups from the Longhorn UI (Volume > Create Backup) or schedule recurring backups. Longhorn backups are incremental — only changed blocks are uploaded after the initial full backup.

Kubernetes Resource Manifests (Configuration Backup)

As an additional safety layer, periodically export all Kubernetes resource definitions. This allows you to recreate resources even without an etcd snapshot, though you would lose any state not captured in the manifests (such as dynamically generated secrets).

```
# Export all resources across all namespaces
kubectl get
all,configmaps,secrets,ingresses,pvc,sc,serviceaccounts,clusterroles,clusterrolebindings
\
  -A -o yaml > ~/k3s-backups/cluster-resources-$(date +%Y%m%d).yaml

# Export just the critical namespaces
for ns in default ingress-nginx longhorn-system metallb-system portainer monitoring;
do
  kubectl -n ${ns} get all,configmaps,secrets,pvc -o yaml > \
    ~/k3s-backups/ns-${ns}-$(date +%Y%m%d).yaml
done
```

Recommended Backup Schedule

etcd snapshots: Automated every 6 hours (already configured). Take a manual snapshot before any cluster changes. Copy snapshots off-cluster weekly.

Longhorn volume snapshots: Daily for active volumes. Configure recurring snapshots in the Longhorn UI under Volume > Recurring Job.

Longhorn external backups: Weekly to S3 or NFS for disaster recovery. Test restoration periodically.

Resource manifests: Weekly export. Store in version control (git) if possible.

13.7 Uninstalling K3s

If you need to completely remove K3s and start over:

```
# On server nodes
/usr/local/bin/k3s-uninstall.sh

# On worker nodes
/usr/local/bin/k3s-agent-uninstall.sh
```

This removes K3s binaries, systemd services, CNI configs, iptables rules, and data directories. It does not affect the OS, NVMe partitions, or other installed packages.

14. Architecture Diagram

```
IPv4 VIP: 192.168.4.100
(kube-vip floating, IPv4 only)
|
+---------------+---------------+
| | |
+-----------+ +-----------+ +-----------+ +-----------+
| k3-node1 | | k3-node2 | | k3-node3 | | k3-node4 |
| Server | | Server | | Server | | Agent |
| etcd | | etcd | | etcd | | |
| | | | | | | |
| .101 | | .102 | | .103 | | .104 |
| ::101 | | ::102 | | ::103 | | ::104 |
| 16GB RAM | | 16GB RAM | | 16GB RAM | | 16GB RAM |
| 1TB NVMe | | 1TB NVMe | | 1TB NVMe | | 1TB NVMe |
+-----------+ +-----------+ +-----------+ +-----------+
Workloads scheduled on ALL 4 nodes
Cluster CIDRs:
Pod IPv4: 10.42.0.0/16
Pod IPv6: fd10:42::/56
Service IPv4: 10.43.0.0/16
Service IPv6: fd10:43::/112
MetalLB Pools:
IPv4: 192.168.4.200 - 192.168.4.220
IPv6: fd00::200 - fd00::220
```